

Lab 08: VLANs in OMNeT++

Duration: 1 hour 30 minutes | No In-Lab Assessment

Tools: OMNeT++ and INET Framework

Objectives:

- Understand VLAN tagging vs. stripping and how INET models VLAN behavior.
- Configure `VlanPolicyLayer` and `Ieee8021qProtocol` to map/accept VLANs per interface.
- Use `Ipv4NetworkConfigurator` to auto-assign addresses for VLAN-separated subnets.
- Build a simple multi-VLAN topology, run a TCP flow (`TcpSessionApp/TcpEchoApp`), and observe L2 isolation vs. cross-VLAN connectivity.
- Practice basic troubleshooting: inspect assigned IPs, check VLAN mappings, and reason about why cross-VLAN traffic is blocked (and how to enable routing).

Short theory:

The lab focuses on Layer-2 VLAN handling inside INET/OMNeT++. Review previous labs on Ethernet and addressing first. Here we exercise how VLAN tags are inserted/parsed, how ports enforce access/trunk behavior, and how VLAN isolation prevents L2 forwarding between VLANs without an L3 router.

Important INET/OMNeT++ concepts for VLANs:

- `VlanPolicyLayer` — maps untagged frames to VLAN IDs, controls inbound/outbound accept lists and mapped VLAN translation.
- `Ieee8021qProtocol` — inserts/parses the 4-byte 802.1Q tag (TPID 0x8100 + PCP/VID) in Ethernet frames.
- Access vs. trunk ports — access ports typically accept an untagged frame and map it to a single VLAN; trunk ports carry multiple VLANs and keep tags intact.
- `Ipv4NetworkConfigurator` — auto-assigns IPv4 addresses per interface/class and fills routing information if configured.
- OMNeT++ inspector — use it to view module parameters and the interfaces' assigned IPs at runtime.

Before the lab

- OMNeT++ and INET installed and built.
- Know how to open NED files and `omnetpp.ini` in the IDE.
- Be ready to inspect module parameters in the runtime inspector (right-click module → Inspect).
- If something fails, consult your INET version's API (module/parameter names can change between releases).

Scenario overview

Two switches connect three department VLANs: EEE (VLAN 10), ME (VLAN 20), AEE (VLAN 30). Hosts plug into `switch1` on access ports bound to their VLAN; `switch1` uplinks to `switch2` via a trunk carrying all VLANs. IPs are auto-assigned per VLAN subnet. A single TCP flow is configured: `eeeHost1` sends 1 MiB to `meHost2` (echo server on port 1000). With only L2 switching and VLAN isolation, this cross-VLAN flow will be blocked — useful to demonstrate VLAN isolation.

Network topology (net.ned)

```

1 01 package inet.examples.ethernet.vlan.lab5;
2 02
3 03 import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
4 04 import inet.node.ethernet.Eth100M;
5 05 import inet.node.ethernet.EthernetSwitch;
6 06 import inet.node.inet.StandardHost;
7 07
8 08 network UniversityVlan
9 09 {
10 10     @display("bgb=968,419");
11 11     submodules:
12 12         configurator: Ipv4NetworkConfigurator { @display("p
13 13             =100,100;is=s"); }
14 14         eeeHost1: StandardHost { @display("p=250,100"); }
15 15         switch1: EthernetSwitch { @display("p=478,99"); }
16 16         switch2: EthernetSwitch { @display("p=687,99"); }
17 17         eeeHost2: StandardHost { @display("p=250,195"); }
18 18         meHost1: StandardHost { @display("p=302,259"); }
19 19         meHost2: StandardHost { @display("p=424,259"); }
20 20         aeeHost1: StandardHost { @display("p=527,259"); }
21 21         aeeHost2: StandardHost { @display("p=632,259"); }
22 22     connections:
23 23         eeeHost1.ethg++ <--> Eth100M <--> switch1.ethg++;
24 24         eeeHost2.ethg++ <--> Eth100M <--> switch1.ethg++;
25 25         meHost1.ethg++ <--> Eth100M <--> switch1.ethg++;
26 26         meHost2.ethg++ <--> Eth100M <--> switch1.ethg++;
27 27         aeeHost1.ethg++ <--> Eth100M <--> switch1.ethg++;
28 28         aeeHost2.ethg++ <--> Eth100M <--> switch1.ethg++;
29 29         switch1.ethg++ <--> Eth100M <--> switch2.ethg++;
30 }

```

Line highlights

- 3–6: import INET building blocks (IPv4 configurator, 100Mb links, switches, standard hosts).
- 8: top-level network `UniversityVlan`.
- 12–20: submodule placement (display coordinates are for the canvas only).
- 22–27: attach each host to `switch1` with 100 Mb links.

- 28: trunk link between switch1 and switch2.

Address plan (net.xml)

```

1 01 <config>
2 02     <interface hosts='eeeHost*' address='192.10.10.x' netmask='
    255.255.255.224' />
3 03     <interface hosts='meHost*' address='192.10.20.x' netmask='
    255.255.255.224' />
4 04     <interface hosts='aeeHost*' address='192.10.30.x' netmask='
    255.255.255.224' />
5 05 </config>

```

Explanation

- All EEE hosts: 192.10.10.0/27.
- All ME hosts: 192.10.20.0/27.
- All AEE hosts: 192.10.30.0/27.
- The wildcard eeeHost* matches instances; the configurator fills per-host last octets.

Simulation settings (omnetpp.ini)

```

1 01 [General]
2 02 network = UniversityVlan
3 03 sim-time-limit = 200s
4 04
5 05 *.*.bridging.typename = "BridgingLayer"
6 06 *.*.bridging.macTableModule = ""
7 07 *.switch*.bridging.directionReverser.reverser.forwardPcp = false
8 08
9 09 *.eeeHost*.bridging.directionReverser.typename = ""
10 10 *.eeeHost*.bridging.interfaceRelay.typename = ""
11 11 *.meHost*.bridging.directionReverser.typename = ""
12 12 *.meHost*.bridging.interfaceRelay.typename = ""
13 13 *.aeeHost*.bridging.directionReverser.typename = ""
14 14 *.aeeHost*.bridging.interfaceRelay.typename = ""
15 15
16 16 *.*.bridging.vlanPolicy.typename = "VlanPolicyLayer"
17 17 *.*.ieee8021q.typename = "Ieee8021qProtocol"
18 18
19 19 *.eeeHost*.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"
    eth0" : {"-1" : 10}}
20 20 *.eeeHost*.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"
    eth0" : [-1, 10]}
21 21 *.eeeHost*.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"
    eth0" : [10]}
22 22
23 23 *.meHost*.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0
    " : {"-1" : 20}}
24 24 *.meHost*.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"
    eth0" : [-1, 20]}

```

```

25 25 *.meHost*.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"
    eth0" : [20]}
26 26
27 27 *.aeeHost*.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"
    eth0" : {"-1" : 30}}
28 28 *.aeeHost*.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"
    eth0" : [-1, 30]}
29 29 *.aeeHost*.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"
    eth0" : [30]}
30 30
31 31 *.switch1.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {
32     "eth0" : [10],
33     "eth1" : [10],
34     "eth2" : [20],
35     "eth3" : [20],
36     "eth4" : [30],
37     "eth5" : [30],
38     "eth6" : [10,20,30]}
39 39
40 40 *.switch1.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {
41     "eth0" : [10],
42     "eth1" : [10],
43     "eth2" : [20],
44     "eth3" : [20],
45     "eth4" : [30],
46     "eth5" : [30],
47     "eth6" : [10,20,30]}
48 48
49 49 *.switch2.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {
50     "eth0" : [10,20,30]}
51 51
52 52 *.configurator.config = xmldoc("net.xml")
53 53
54 54 **.fcsMode = "computed"
55 55 **.crcMode = "computed"
56 56
57 57 **.*.llc.typename = ""
58 58 **.*.ethernet.typename = "EthernetLayer"
59 59 **.*.eth[*].typename = "LayeredEthernetInterface"
60 60 **.*.eth[*].bitrate = 100Mbps
61 61
62 62 *.eeeHost1.numApps = 1
63 63 *.eeeHost1.app[*].typename = "TcpSessionApp"
64 64 *.eeeHost1.app[*].connectAddress = "meHost2"
65 65 *.eeeHost1.app[*].connectPort = 1000
66 66 *.eeeHost1.app[*].sendBytes = 1MiB
67 67 *.eeeHost1.app[*].tOpen = 0s
68 68 *.eeeHost1.app[*].tSend = 0s
69 69
70 70 *.meHost2.numApps = 1
71 71 *.meHost2.app[*].typename = "TcpEchoApp"
72 72 *.meHost2.app[0].localAddress = ""
73 73 *.meHost2.app[0].localPort = 1000

```

Line-by-line explanation

- 1–3: set network and simulation time limit.
- 5: use `BridgingLayer` everywhere to model L2 behavior.
- 6: disable external MAC-table module; use built-in behavior.
- 9–14: hosts disable direction-reverser and interface-relay helpers (simpler Ethernet stack).
- 16–17: enable `VlanPolicyLayer` and `Ieee8021qProtocol` to implement VLAN tagging/acceptance.
- 19–29: per-host mapping and filters — untagged egress (-1) is mapped to the host's VLAN (10/20/30); ingress accepts untagged or that VLAN; outbound allows only the VLAN (so egress frames leave tagged).
- 31–38 / 40–47: `switch1` inbound/outbound filters classify access ports (eth0/1 EEE, eth2/3 ME, eth4/5 AEE) and eth6 as trunk (accepts all VLANs).
- 49–50: `switch2` trunk accepts all VLANs.
- 52: configurator pointed at `net.xml` for address assignment.
- 54–55: compute FCS/CRC rather than assume them present.
- 57–60: minimal Ethernet stack and 100 Mbps interfaces.
- 62–69: client on `eeeHost1` opens and sends 1 MiB to `meHost2:1000`.
- 70–73: echo server on `meHost2` listening on TCP port 1000.

How tagging behaves in this scenario

- Hosts start with untagged frames; the outbound mapper replaces VLAN -1 with 10/20/30 so frames exit the NIC tagged.
- Access ports on `switch1` reject VLANs not allowed on that port, preventing inter-VLAN leakage at the edge.
- The trunk between `switch1` and `switch2` carries tags intact; neither side strips tags.
- No VLAN → -1 mapping is configured, therefore frames remain tagged end-to-end. Stripping would require an explicit VLAN→-1 mapping on the egress interface.

End-to-end flow walkthrough

1. `eeeHost1` creates a TCP packet; the outbound mapper tags it with VLAN 10.
2. `switch1` receives on eth0/1, confirms VLAN 10 is allowed, forwards the frame via trunk eth6.

3. `switch2` accepts VLAN 10 on its trunk and forwards according to its MAC table (or back across the network toward VLAN 10 hosts).
4. VLAN separation prevents delivery to ME ports: VLAN 10 frames will not egress on ports configured for VLAN 20/30. Cross-VLAN traffic requires L3 routing (router-on-a-stick or an L3 interface per VLAN).

Note The supplied configuration intentionally configures a cross-VLAN client/server pair. With only L2 switching and VLAN isolation, `eeeHost1` cannot reach `meHost2`. This demonstrates L2 isolation. To enable reachability, add a router with sub-interfaces (or a Layer-3 switch) and route between VLANs.

Troubleshooting checklist

- Inspect host interface parameters in Qtenv (right-click → Inspect) to confirm the VLAN mapper and `Ieee8021qProtocol` are present.
- Check that `outboundMapper` maps `-1` to the intended VLAN (e.g., `"-1":10`).
- Verify switch port filters: inbound/outbound accept lists must include the intended VLAN on each port.
- If frames appear only as ARP requests and no TCP, confirm IP addresses were assigned and the destination resolves to the correct MAC.
- To test stripping, add a mapping that translates `VLAN→-1` on the egress interface and observe tag removal.

Optional extension

Add a router (or a host acting as router) with subinterfaces for VLANs (router-on-a-stick) and configure static routes (or enable `Ipv4NetworkConfigurator` to add routes) to demonstrate inter-VLAN routing and successful end-to-end TCP transfer.